

Spring Boot 인증 서버 발표

- 주제: JWT + OAuth2 기반 인증 백엔드 설계
- 설명 포인트: 구조, 핵심 함수 역할, 이렇게 만든 이유
- 핵심 가치: 확장성, 보안성, 설명 가능한 코드

프로젝트 개요

- 로컬 회원가입과 로그인 지원
- JWT access token 발급과 인증 유지
- Google, Kakao, Naver OAuth2 로그인 지원
- 전역 예외 처리와 요청 검증까지 포함한 인증 서버

왜 이 구조를 선택했는가

- controller는 HTTP 입출력만 담당해 책임을 단순화했습니다.
- service는 회원가입과 로그인 비즈니스 로직을 담당합니다.
- repository는 DB 접근을 분리해 테스트와 유지보수를 쉽게 합니다.
- security/global 패키지로 인증과 공통 예외 처리를 모았습니다.

시스템 구조

- AuthController -> AuthService -> UserRepository 흐름으로 기본 인증 로직을 구성했습니다.
- SecurityConfig가 필터 체인, CORS, OAuth2 로그인 설정을 통합합니다.
- JwtFilter와 JwtUtil이 토큰 검증과 인증 객체 주입을 담당합니다.

핵심 엔티티와 함수

- User는 이메일, 비밀번호, provider, providerId를 함께 저장합니다.
- signup()은 중복 이메일 검사 후 비밀번호를 BCrypt로 암호화합니다.
- login()은 로컬 계정 여부와 비밀번호 일치 여부를 확인한 뒤 JWT를 발급합니다.
- provider 검사로 소셜 계정의 비밀번호 로그인 시도를 막습니다.

로컬 로그인 처리 흐름

- 클라이언트가 /api/auth/login 요청을 보냅니다.
- 서비스 계층에서 사용자 조회와 provider 검사를 수행합니다.
- BCrypt로 비밀번호 해시를 비교한 뒤 JwtUtil로 토큰을 생성합니다.
- 세션 대신 토큰을 반환해 API 서버 구조를 단순하게 유지합니다.

OAuth2 로그인 처리 흐름

- Spring Security가 소셜 로그인 흐름을 시작합니다.
- CustomOAuth2UserService가 provider별 사용자 정보를 공통 형식으로 변환합니다.
- 기존 회원은 갱신하고 신규 회원은 생성하는 upsert 전략을 사용합니다.
- 성공 시 JWT를 발급해 프론트 콜백 URL로 리다이렉트합니다.

보안 설계 이유

- 비밀번호는 평문 저장 대신 BCrypt 해시로 관리합니다.
- JwtFilter는 매 요청마다 Bearer 토큰을 검증합니다.
- 로그인 실패 메시지를 통일해 계정 존재 여부를 노출하지 않습니다.
- GlobalExceptionHandler로 에러 응답 형식을 일관되게 유지합니다.

예외 처리와 테스트 전략

- 입력 검증 실패와 비즈니스 예외를 한곳에서 처리합니다.
- AuthServiceTest가 로그인/회원가입 핵심 로직을 검증합니다.
- AuthControllerApiTest가 HTTP 상태 코드와 JSON 응답 형식을 검증합니다.
- JWT와 OAuth2 테스트로 보안 흐름의 회귀를 방지합니다.

시연 포인트와 마무리

- 회원가입 -> 로그인 -> access token 확인 순서로 시연합니다.
- Authorization 헤더로 /api/auth/me 호출 결과를 보여줍니다.
- 소셜 로그인 URL과 구조 설명으로 확장성을 강조합니다.
- 핵심 메시지: 기능 구현보다 설명 가능한 구조 설계에 초점을 맞췄습니다.